Food Delivery Data Analysis Using Python

Project Overview

This project performs Exploratory Data Analysis (EDA) on food delivery order data to understand order patterns, vendor revenue, delivery performance, and customer ratings.

Objectives

- Analyze order distribution across cities.
- Identify top-performing vendors by order value.
- Study delivery time and distance patterns.
- Investigate delivery delays and cancellations.
- Explore monthly order value trends.

Tools Used

Python, Pandas, Seaborn, Matplotlib, Jupyter Notebook

Dataset

3,000 records and 17 original columns.

EDA - Exploratory Data Analysis

- it used for analysing data before importing file

1. Importing Libery/Modules

In [1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

2. Importing File and crate Data Frame (file load)

In [2]:
```
try:
    df = pd.read_csv(r"C:\Users\janha\OneDrive\Desktop\Python EDA\delivery_order
    print("File loaded successfully")
```

```
except FileNotFoundError:
    print("CSV file not found. Check the file location.")
```

```
File loaded successfully
```

3. Insepecting File

3.1 Show info of data frame (check for data type)

In [62]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 21 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   order_id           3000 non-null   str
 1   order_date         3000 non-null   datetime64[us]
 2   order_placed_time  3000 non-null   str
 3   day_of_week        3000 non-null   str
 4   city               3000 non-null   str
 5   vendor_id          3000 non-null   str
 6   vendor_name        3000 non-null   str
 7   order_type         3000 non-null   str
 8   driver_id          3000 non-null   str
 9   distance_km        3000 non-null   float64
 10  weather_condition  3000 non-null   str
 11  is_peak_hour       3000 non-null   int64
 12  order_value_inr    3000 non-null   float64
 13  delivery_time_min  3000 non-null   float64
 14  is_delayed         3000 non-null   object
 15  is_cancelled       3000 non-null   object
 16  customer_rating    3000 non-null   float64
 17  deliver_remark     3000 non-null   str
 18  distance_remark    3000 non-null   str
 19  Day                3000 non-null   int32
 20  Month              3000 non-null   int32
dtypes: datetime64[us](1), float64(4), int32(2), int64(1), object(2), str(11)
memory usage: 468.9+ KB
```

3.2 are there any duplicate value present

In [63]:
```
# auto comand if it has duplicate it auto remove duplicate
if df.duplicated().sum()>0:
    print("duplicat record found",df.duplicated().sum())
    df.drop_duplicates(inplace=True)
else:
    print("No Duplicates Found")
```

```
No Duplicates Found
```

In [64]:
```
dup = df.duplicated().sum()
print("Duplicate records in dataset =",dup)
```

```
Duplicate records in dataset = 0
```

3.3 Check There are any null values present

In [65]:
```
# Check missing values
df.isnull().sum()
```

Out[65]:
```
order_id             0
order_date           0
order_placed_time    0
day_of_week          0
city                 0
vendor_id            0
vendor_name          0
order_type           0
driver_id            0
distance_km          0
weather_condition    0
is_peak_hour         0
order_value_inr      0
delivery_time_min    0
is_delayed           0
is_cancelled         0
customer_rating      0
deliver_remark       0
distance_remark      0
Day                  0
Month                0
dtype: int64
```

In [66]:
```
#find Average of value

avg = df["delivery_time_min"].mean().round()
print("aveage delivery time is =", avg)
df["delivery_time_min"] = df["delivery_time_min"].fillna(avg)
```

```
aveage delivery time is = 31.0
```

In [67]:
```
df["delivery_time_min"]
```

Out[67]:
```
0       27.5
1       21.0
2       16.1
3       41.3
4       24.7
        ...
2995    28.6
2996    36.1
2997    26.1
2998    17.9
2999    19.7
Name: delivery_time_min, Length: 3000, dtype: float64
```

3.4 Show rows and column count

In [68]:
```
count = df.shape
print("Row Count =",count[0] )
print("Column count =",count[1])
```

```
Row Count = 3000
Column count = 21
```

3.1.1 Changing data type str -> date time

In [69]:
```
df["order_date"] = pd.to_datetime(df["order_date"])
```

In [70]:
```
df["order_date"]
```

Out[70]:
```
0      2026-01-03
1      2026-02-07
2      2026-04-08
3      2026-04-20
4      2026-03-10
          ...
2995   2026-06-22
2996   2026-05-15
2997   2026-02-05
2998   2026-04-22
2999   2026-05-25
Name: order_date, Length: 3000, dtype: datetime64[us]
```

In [71]:
```
most_frquent_rating = df["customer_rating"].mode()[0]
df["customer_rating"] = df["customer_rating"].fillna(most_frquent_rating)
```

4. Univeriant Analysis

1. show total order count
2. show total vendors register on platform
3. show avg order price
4. show minimum,maximum,average delivery time
5. show avg distancein km

In [72]:
```
order = df["order_id"].count().sum()
print("total order count is =", order)
```

```
total order count is = 3000
```

In [73]:
```
vendor = df["vendor_id"].count().sum()
print("total vendor register on platform =",vendor)
```

```
total vendor register on platform = 3000
```

In [74]:
```
avg_price = df["order_value_inr"].mean().round()
print("average order price is =", avg_price)
```

```
average order price is = 763.0
```

In [75]:
```
dtime = df["delivery_time_min"].max()
print("maximum deliver time in minutes =",dtime)
```

```
maximum deliver time in minutes = 109.7
```

In [76]:
```
dtime = df["delivery_time_min"].min()
print("minimum deliver time in minutes =",dtime)
```

```
minimum deliver time in minutes = 6.9
```

In [77]:
```
dtime = df["delivery_time_min"].mean().round()
print("Average deliver time in minutes =",dtime)
```

```
Average deliver time in minutes = 31.0
```

In [78]:
```
avg_distance = df["distance_km"].mean().round()
print("Average Distace in KM is =",avg_distance)
```

```
Average Distace in KM is = 3.0
```

Data Visualization in Python

- Graphical Representation of Numeric Data

In [79]:
```
df
```

Out[79]:

	order_id	order_date	order_placed_time	day_of_week	city	vendor_id	ve
0	ORD00001	2026-01-03	22:33:00	Saturday	Mumbai	V007	W
1	ORD00002	2026-02-07	16:01:00	Saturday	Delhi	V006	
2	ORD00003	2026-04-08	16:56:00	Wednesday	Chennai	V002	
3	ORD00004	2026-04-20	22:38:00	Monday	Chennai	V001	
4	ORD00005	2026-03-10	22:50:00	Tuesday	Chennai	V001	
...	...	...	...	...	...	...	
2995	ORD02996	2026-06-22	14:37:00	Monday	Bengaluru	V004	C
2996	ORD02997	2026-05-15	18:51:00	Friday	Chennai	V001	
2997	ORD02998	2026-02-05	14:28:00	Thursday	Bengaluru	V008	Pi
2998	ORD02999	2026-04-22	15:56:00	Wednesday	Chennai	V002	
2999	ORD03000	2026-05-25	15:22:00	Monday	Bengaluru	V008	Pi

3000 rows × 21 columns

5. Adding New Columns

In [80]:
```
def remark(time):
    if time > 60:
        return("very late")
    elif time >30:
        return("Late")
    elif time >10:
        return("on Time")
    else:
        return("very fast")
```

In [81]:
```
remark(90)
```

Out[81]: 'very late'

In [82]:
```
df["deliver_remark"] = df["delivery_time_min"].map(remark)
```

In [83]:
```
df.groupby("deliver_remark")["order_id"].count().reset_index()
```

Out[83]:

	deliver_remark	order_id
0	Late	1218
1	on Time	1672
2	very fast	13
3	very late	97

In [84]:
```
df.shape
```

Out[84]: (3000, 21)

In [85]:
```
def distance(km):
    if km > 10:
        return("long Didtance")
    elif km >5 :
        return("Short Distance")
    else:
        return("Near")
```

In [86]:
```
df["distance_remark"] = df["distance_km"].map(distance)
```

In [87]:
```
df.groupby("distance_remark")["order_id"].count().reset_index()
```

Out[87]:

	distance_remark	order_id
0	Near	2498
1	Short Distance	475
2	long Didtance	27

6. Bi - varient Analysis and Visualization

6.1 City by order count

In [88]:
```
df.columns
```

Out[88]:
```
Index(['order_id', 'order_date', 'order_placed_time', 'day_of_week', 'city',
       'vendor_id', 'vendor_name', 'order_type', 'driver_id', 'distance_km',
       'weather_condition', 'is_peak_hour', 'order_value_inr',
       'delivery_time_min', 'is_delayed', 'is_cancelled', 'customer_rating',
       'deliver_remark', 'distance_remark', 'Day', 'Month'],
      dtype='str')
```

In [89]:
```
city_order = df.groupby("city")["order_id"].count().sort_values().reset_index()
city_order
```

Out[89]:

	city	order_id
0	Delhi	475
1	Mumbai	501
2	Hyderabad	509
3	Chennai	736
4	Bengaluru	779

In [90]:
```
# how to ceate column chart in  python
```

In [91]:
```
# customization

plt.figure(figsize=(10, 5))

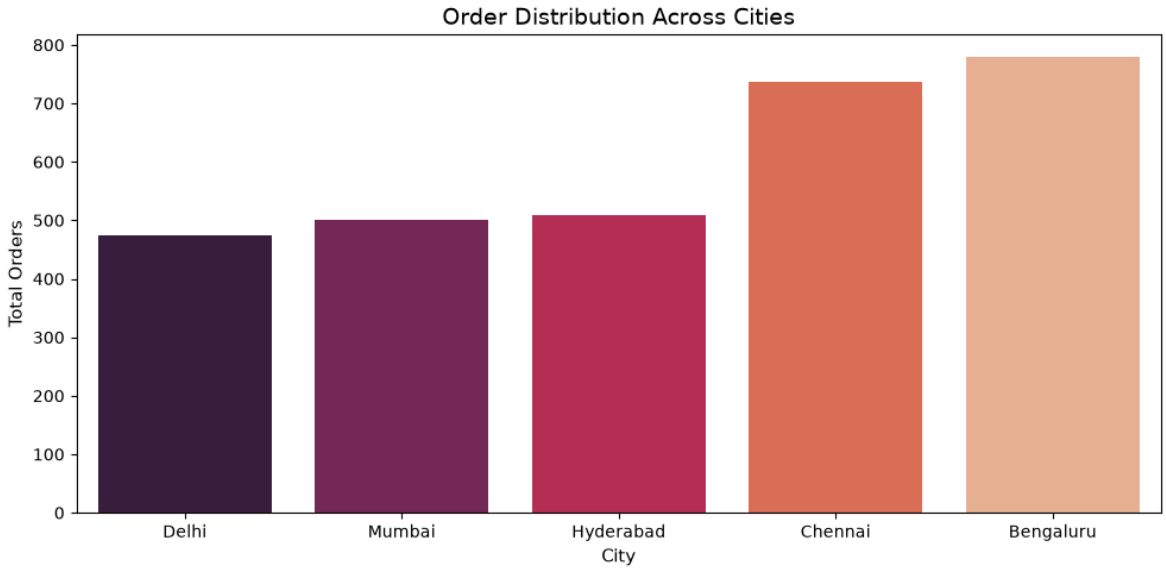
sns.barplot(x="city",y="order_id",data=city_order,palette="rocket")

plt.title("Order Distribution Across Cities", fontsize=14)
plt.xlabel("City", fontsize=11)
plt.ylabel("Total Orders", fontsize=11)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
C:\Users\janha\AppData\Local\Temp\ipykernel_6068\2798003547.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x="city",y="order_id",data=city_order,palette="rocket")
```



6.2 Vendors by revenue

In [92]:
```
vendor_revenue=df.groupby('vendor_name') ["order_value_inr"].sum().round().sort_
```

In [93]:
```
vendor_revenue
```

Out[93]:

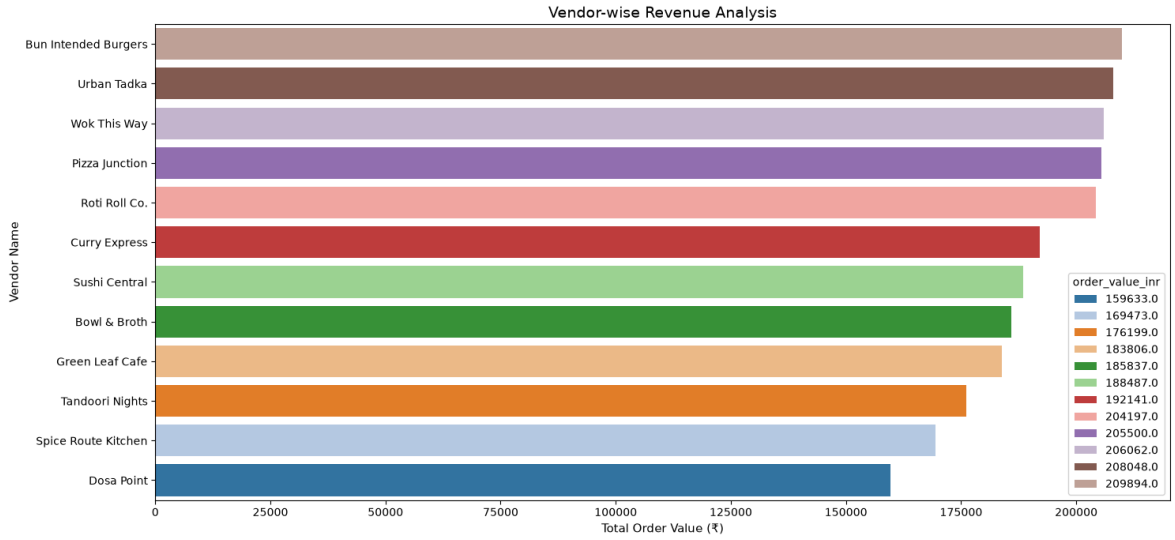
	vendor_name	order_value_inr
0	Bun Intended Burgers	209894.0
1	Urban Tadka	208048.0
2	Wok This Way	206062.0
3	Pizza Junction	205500.0
4	Roti Roll Co.	204197.0
5	Curry Express	192141.0
6	Sushi Central	188487.0
7	Bowl & Broth	185837.0
8	Green Leaf Cafe	183806.0
9	Tandoori Nights	176199.0
10	Spice Route Kitchen	169473.0
11	Dosa Point	159633.0

In [94]:
```
plt.figure(figsize=(15,7))

sns.barplot(y="vendor_name", x="order_value_inr", data= vendor_revenue, palette=

plt.title("Vendor-wise Revenue Analysis", fontsize=14)
plt.xlabel("Total Order Value (₹)", fontsize=11)
plt.ylabel("Vendor Name", fontsize=11)
plt.tight_layout()
```

```
plt.show()
#hue is given for Legends
```



In [95]:
```
df ["is_delayed"]=df ["is_delayed"].replace(1, "Yes").replace(0, "No")
```

In [96]:
```
df["is_cancelled"]=df["is_cancelled"].replace(0,"No").replace(1, "Yes")
df
```

Out[96]:

	order_id	order_date	order_placed_time	day_of_week	city	vendor_id	ve
0	ORD00001	2026-01-03	22:33:00	Saturday	Mumbai	V007	W
1	ORD00002	2026-02-07	16:01:00	Saturday	Delhi	V006	
2	ORD00003	2026-04-08	16:56:00	Wednesday	Chennai	V002	
3	ORD00004	2026-04-20	22:38:00	Monday	Chennai	V001	
4	ORD00005	2026-03-10	22:50:00	Tuesday	Chennai	V001	
...	...	...	...	...	...	...	...
2995	ORD02996	2026-06-22	14:37:00	Monday	Bengaluru	V004	C
2996	ORD02997	2026-05-15	18:51:00	Friday	Chennai	V001	
2997	ORD02998	2026-02-05	14:28:00	Thursday	Bengaluru	V008	Pi
2998	ORD02999	2026-04-22	15:56:00	Wednesday	Chennai	V002	
2999	ORD03000	2026-05-25	15:22:00	Monday	Bengaluru	V008	Pi

3000 rows × 21 columns

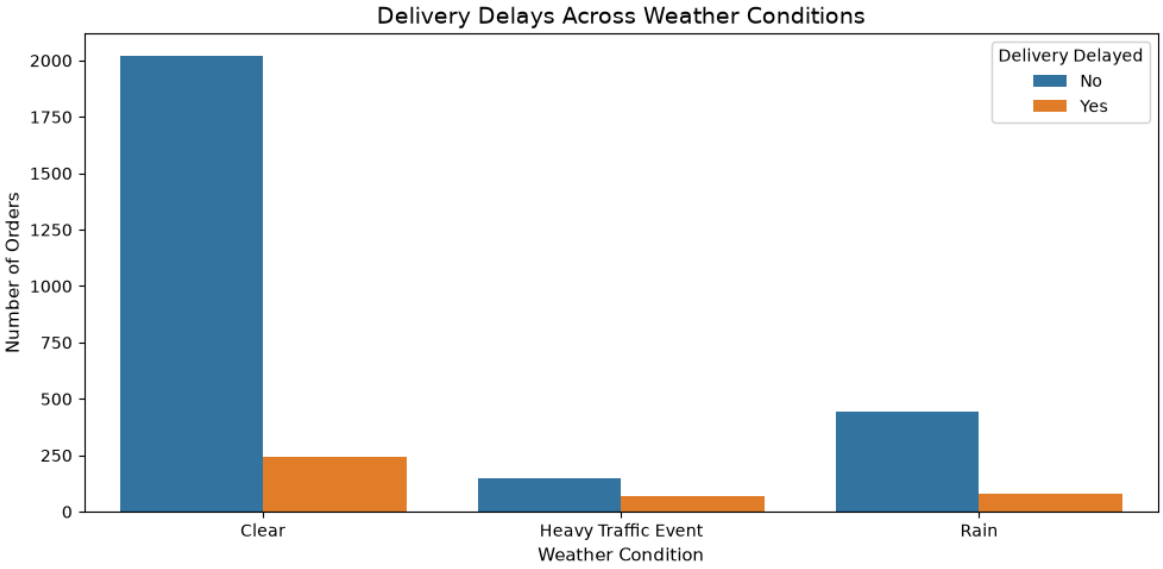
6.3 Column Chart

In [97]:
```
wether_data=df.groupby(["weather_condition", 'is_delayed'])['order_id'].count().
```

In [98]:
```
plt.figure(figsize=(10, 5))

sns.barplot(x="weather_condition", y= 'order_id', data= wether_data, hue='is_del

plt.title("Delivery Delays Across Weather Conditions", fontsize=14)
plt.xlabel("Weather Condition", fontsize=11)
plt.ylabel("Number of Orders", fontsize=11)
plt.legend(title="Delivery Delayed")
plt.tight_layout()
plt.show()
```



6.4 Pie Chart

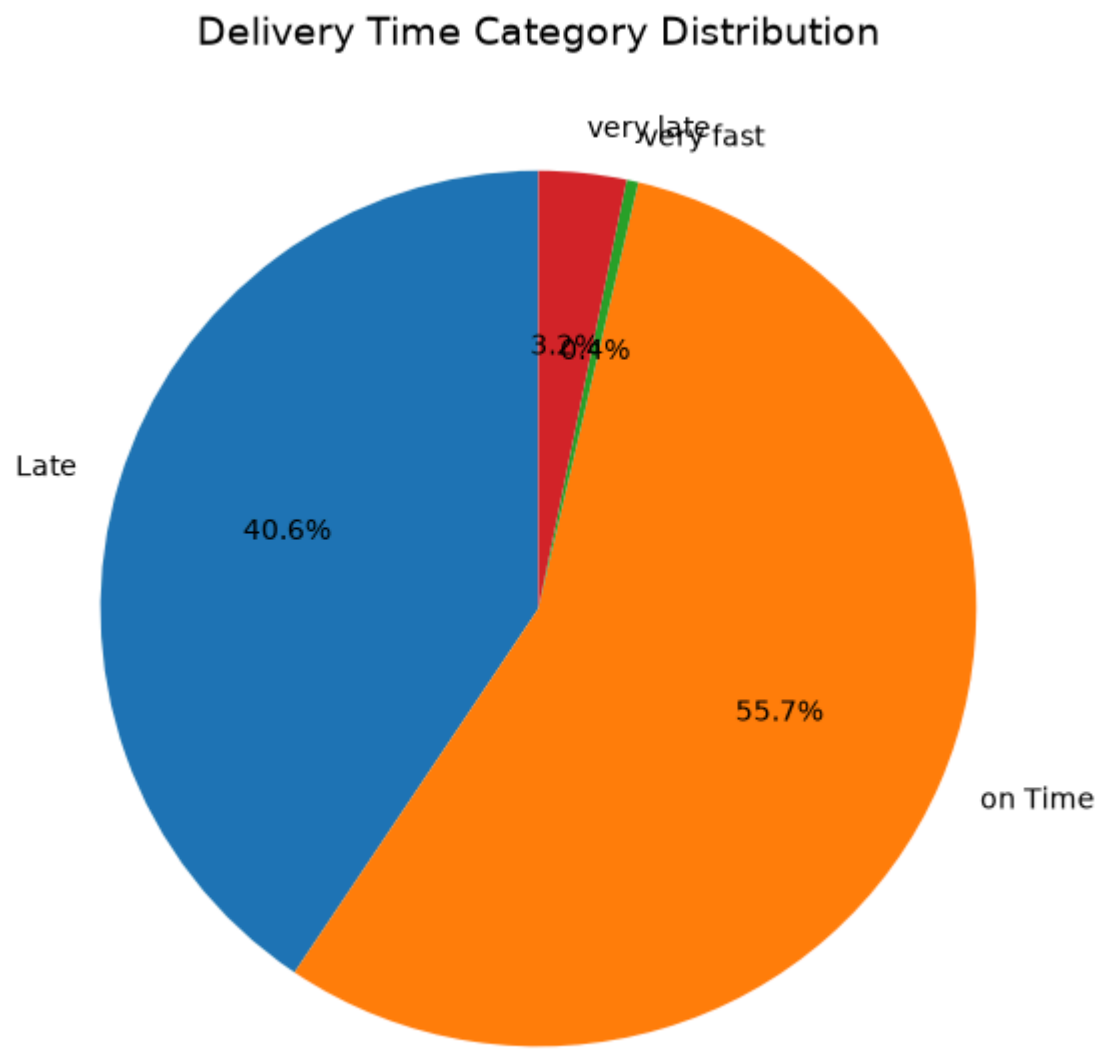
In [99]:
```
deliver_remark= df.groupby("deliver_remark")["order_id"].count()
deliver_remark
```

Out[99]:
```
deliver_remark
Late         1218
on Time      1672
very fast      13
very late      97
Name: order_id, dtype: int64
```

In [100…
```
plt.figure(figsize=(8, 6))

deliver_remark.plot(kind="pie",autopct="%1.1f%%",startangle=90)

plt.title("Delivery Time Category Distribution", fontsize=14)
plt.ylabel("")
plt.tight_layout()
plt.show()
```



6.5 Delivery distance vs delivery time

In [101…
```
df.columns
```

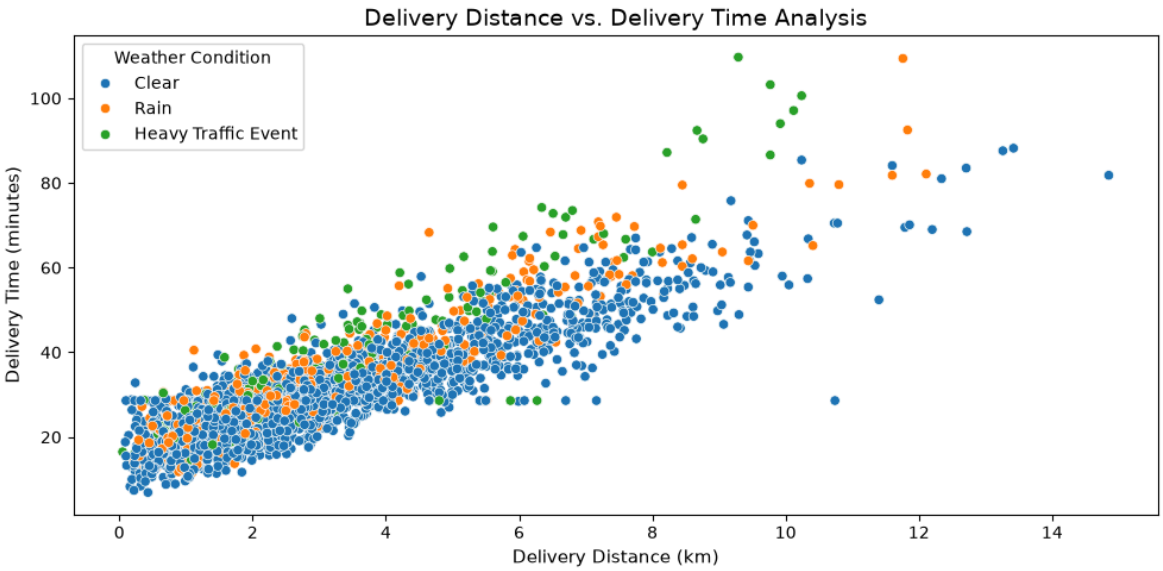
Out[101…
```
Index(['order_id', 'order_date', 'order_placed_time', 'day_of_week', 'city',
       'vendor_id', 'vendor_name', 'order_type', 'driver_id', 'distance_km',
       'weather_condition', 'is_peak_hour', 'order_value_inr',
       'delivery_time_min', 'is_delayed', 'is_cancelled', 'customer_rating',
       'deliver_remark', 'distance_remark', 'Day', 'Month'],
      dtype='str')
```

In [102…
```
plt.figure(figsize=(10, 5))

sns.scatterplot(x="distance_km",y="delivery_time_min",data=df,hue="weather_condi

plt.title("Delivery Distance vs. Delivery Time Analysis", fontsize=14)
plt.xlabel("Delivery Distance (km)", fontsize=11)
plt.ylabel("Delivery Time (minutes)", fontsize=11)
plt.legend(title="Weather Condition")
plt.tight_layout()
plt.show()
```

```
#used scaterplot wherever we need each record
```



In [103…
```
df["Day"] = df["order_date"].dt.day
#extracting day month year from date column
```

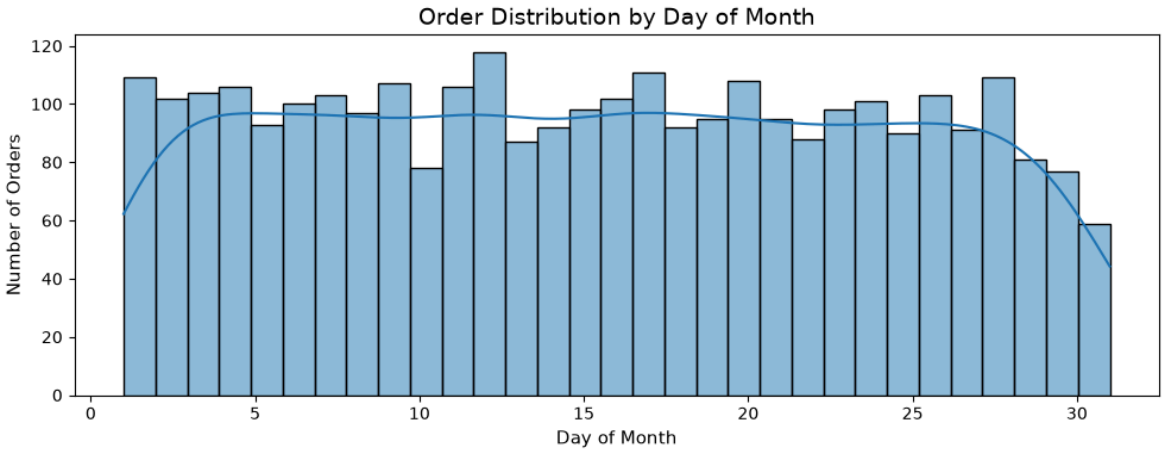
In [104…
```
df["Day"]
```

Out[104…
```
0        3
1        7
2        8
3       20
4       10
        ..
2995    22
2996    15
2997     5
2998    22
2999    25
Name: Day, Length: 3000, dtype: int32
```

6.6 Day by Order Count

In [105…
```
plt.figure(figsize=(10,4))                                    #used for give hi
sns.histplot(x="Day", data= df, bins = 31 , kde=True)          #kde=Trueused fo
                                                               # bins to show r

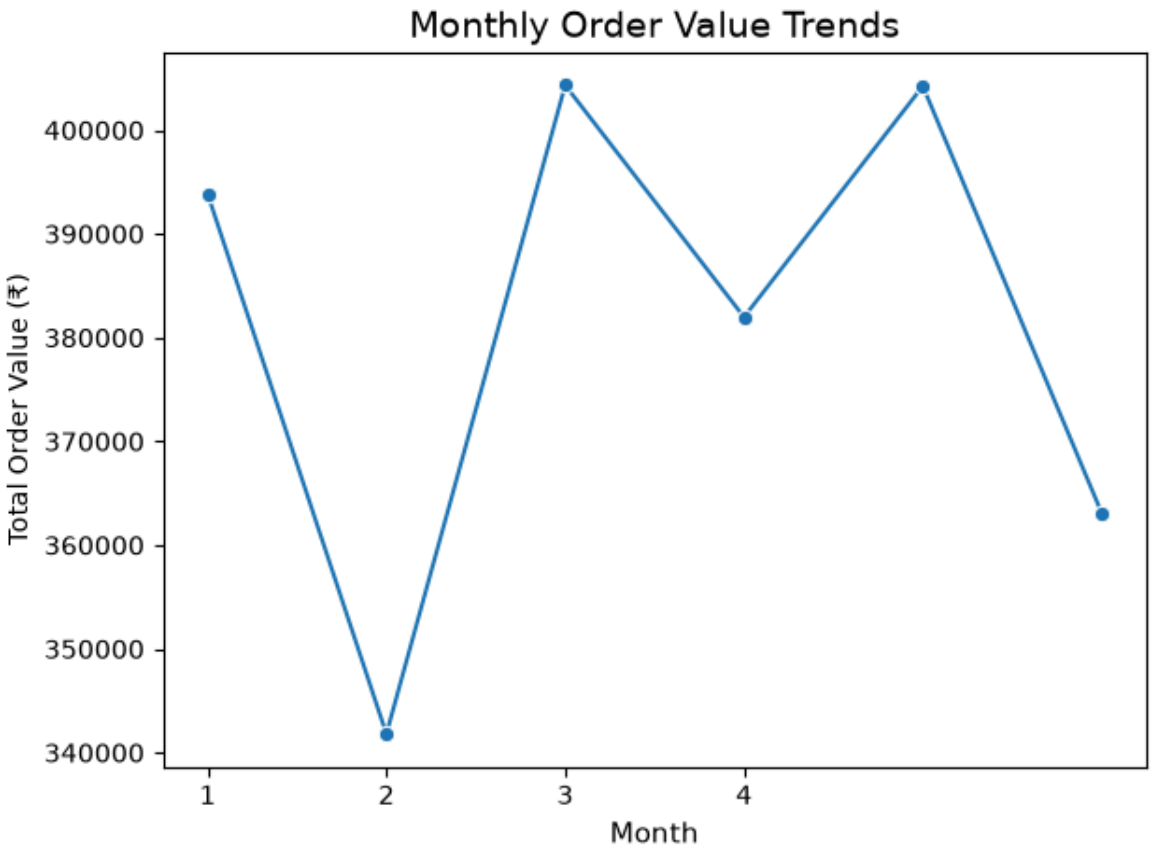

plt.title("Order Distribution by Day of Month", fontsize=14)
plt.xlabel("Day of Month", fontsize=11)
plt.ylabel("Number of Orders", fontsize=11)
plt.tight_layout()
plt.show()
```



In [106…
```
df["Month"] = df["order_date"].dt.month
```

In [107…
```
data = df.groupby("Month")["order_value_inr"].sum().reset_index()
sns.lineplot(x= "Month", y= "order_value_inr", data=data,marker="o")

plt.title("Monthly Order Value Trends", fontsize=14)
plt.xlabel("Month", fontsize=11)
plt.ylabel("Total Order Value (₹)", fontsize=10)
plt.xticks(range(1, 5))
plt.tight_layout()
plt.show()
```



7. Conclusion

In this Exploratory Data Analysis (EDA) project, I analyzed food delivery order data using Python, Pandas, Matplotlib, and Seaborn.

The analysis explored order distribution across cities, vendor-wise revenue, the relationship between weather conditions and delivery delays, delivery time categories, delivery distance versus delivery time, daily order distribution, and monthly order value trends.

This project helped me understand data cleaning, data transformation, data aggregation, and data visualization to identify meaningful patterns in the dataset.

Tools Used: Python, Pandas, Matplotlib, Seaborn

Project Outcome: Developed practical skills in data analysis and visualization to support data-driven decision-making.